

Chapter 4

CPU DESIGN

Contents:

1. Introduction to CPU
2. Specifying a CPU
3. Design and Implementation of a Very Simple and Relatively Simple CPU
4. Instruction Execution, Fetch, Decode , Data path
5. ALU Design
6. Designing Hardwired Control Unit
7. Design Verification

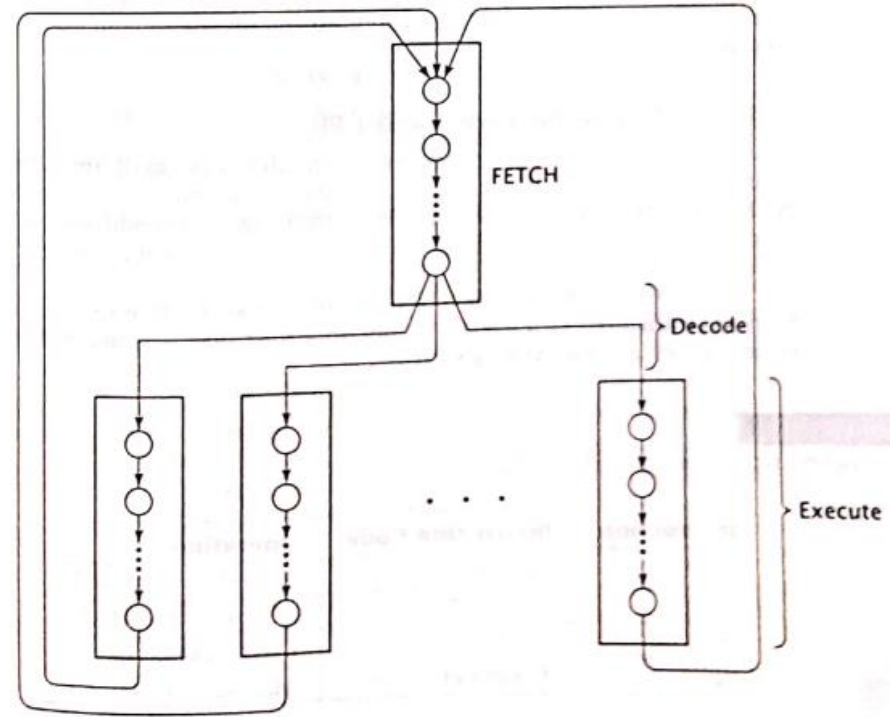
4.1 Introduction to CPU

- Central Processing Unit
- **CPU = ALU + Registers + Control Unit**
- Performs sequences of microoperations needed to perform **fetch**, **decode** and **execute** cycles of every instructions in CPU instruction set.

4.2 Specifying a CPU

- The first step in designing CPU is to **determine its applications**. The key is to match the capabilities of the CPU to the tasks it will perform.
- Second step is to **design a Instruction Set Architecture (ISA)** which can handle the given tasks.
- Third step is to **design State diagram for CPU**. Here the different microoperation are shown, which are performed during each stat and the conditions that cause CPU to go from one state to another. CPU is a complex Finite State Machine.
- Then, specifying the steps the CPU must perform in order to fetch, decode and execute every instructions in its instruction set.

- In general, a CPU performs following sequence of operations.
 1. **Fetch Cycle:** Fetching an instruction from memory, then go to decode cycle.
 2. **Decode Cycle:** Decoding an instruction- that is, determine which instruction has been fetched.
 3. **Execute cycle:** Executing the instruction . Then go to the fetch cycle and fetch the next instruction.



Generic CPU state diagram

4.3 Design and Implementation of a Very Simple and Relatively Simple CPU

- To illustrate the CPU design process,
- lets us consider:
 - A small and somewhat impractical CPU.
 - It can access 64 bytes of memory, each byte being 8 bits wide.
 - The CPU does this by outputting a 6-bit address on its output pins A[5..0], and reading in the 8-bit value from memory on its inputs D[7...0].
 - The CPU has: one programmable register, 8-bit accumulator (AC).
 - It has only four instructions in its instruction set.

Instruction	Instruction Code	Operation
ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	$AC \leftarrow AC + 1$

Instruction set for the Very Simple CPU

Design and Implementation of a Very Simple and Relatively Simple CPU

- In addition to AC, the CPU contains the following registers:
 1. **Address Register (AR):**
 - 6-bit
 - Supplies an address to memory via A[5..0].
 2. **Program Counter(PC):**
 - 6-bit
 - Contains the address of the next instruction to be executed.
 3. **Data Register(DR):**
 - 8-bit
 - Receives instructions and data from memory via D[7...0]
 4. **Instruction Register(IR):**
 - 2-bit
 - Stores the opcode portion of the instruction code fetched from memory.

4.4 Instruction Execution, Fetch, Decode and Datapath

- Fetching instructions from memory
- Decoding instructions
- Executing instructions
- Constructing Datapath

1. Fetching instructions from Memory.

Instruction	Instruction Code	Operation
ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	$AC \leftarrow AC + 1$

- The address of instruction to be fetched is stored in the PC.
- Since, address register (AR) supplies an address to memory through A [5.....0] pins
- Firstly, copy the content of PC to AR. Thus, the first state of the fetch cycle is:

Fetch 1: $AR \leftarrow PC$

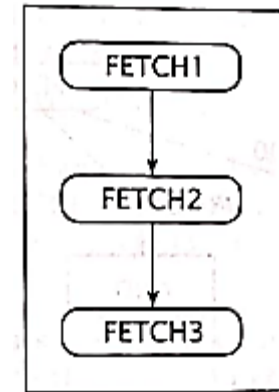
- Next, the CPU reads the instructions from the memory. The CPU must assert READ signal, which is output from the CPU to memory. This causes memory to store in Data Register (DR).

**Fetch 2: $DR \leftarrow M$
 $PC \leftarrow PC + 1$**

- Finally, two higher order bits of DR are copied into Instruction Register (IR); these two bits indicate which instruction is to be executed, and the CPU copies the lower order 6-bit of DR into AR.

**Fetch 3: $IR \leftarrow DR [7,6]$
 $AR \leftarrow DR [5....0]$**

- The state diagram for the fetch cycle is shown aside.

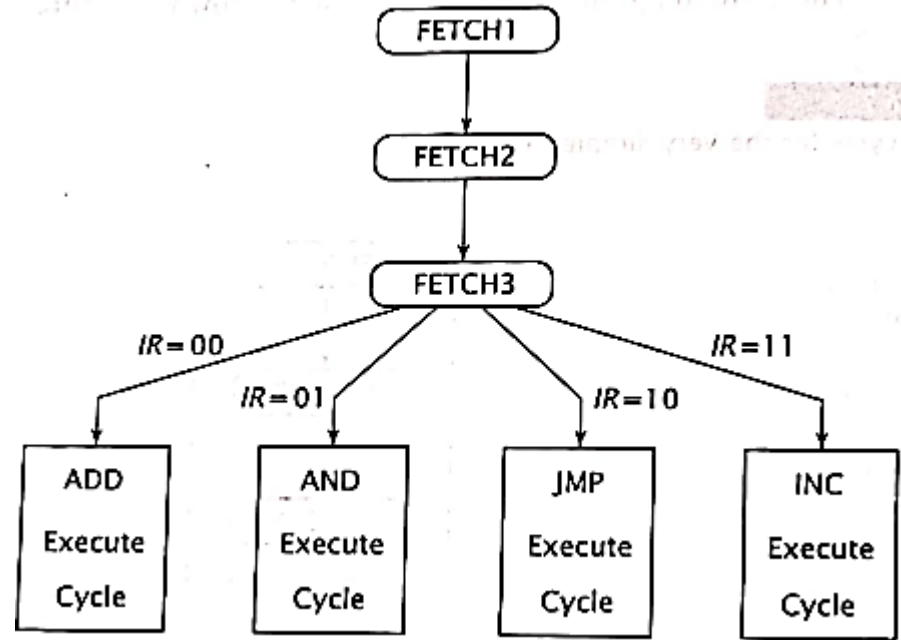


Fetch cycle for the Very Simple CPU

2. Decoding instructions

- After CPU has fetched an instruction from memory, it must determine which instruction it has fetched so that it may invoke the correct **execute routine**.
- For this CPU, there are four instructions and thus, four executer routines.
- The value in IR (00,01,10 or 11) determines which execute routine is invoked.

Instruction	Instruction Code	Operation
ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	$AC \leftarrow AC + 1$



Fetch and decode cycles for the Very Simple CPU

3. Executing instructions

- To complete the state diagram for this CPU, we must **develop the state diagram for each execute routine**.
- Now, we design the portion of the state diagram for each routine and the overall design for the CPU.
- The state diagrams for the individual execute routines are fairly simple, so they are only included in the diagram of the finite state machine for the entire CPU.

a) ADD instruction

- To perform ADD instruction, CPU must :
- First, fetch one operand from memory.
- Then add this operand to current contents in AC and store result back in the AC.

ADD1: $DR \leftarrow M$

ADD2: $AC \leftarrow AC + DR$

b) AND instruction

- Virtually similar to ADD instruction.
- First, fetch an operand from memory.
- Logically AND the two values and store it in AC.

AND1: $DR \leftarrow M$

AND2: $AC \leftarrow AC \wedge DR$

c) JMP instruction

- The address to which the CPU must jump is copied into the PC.
- Then CPU fetches the next instruction.

JMP1: $PC \leftarrow DR[5..0]$

d) INC instruction

- CPU simply adds 1 to contents of AC and goes to fetch routine.

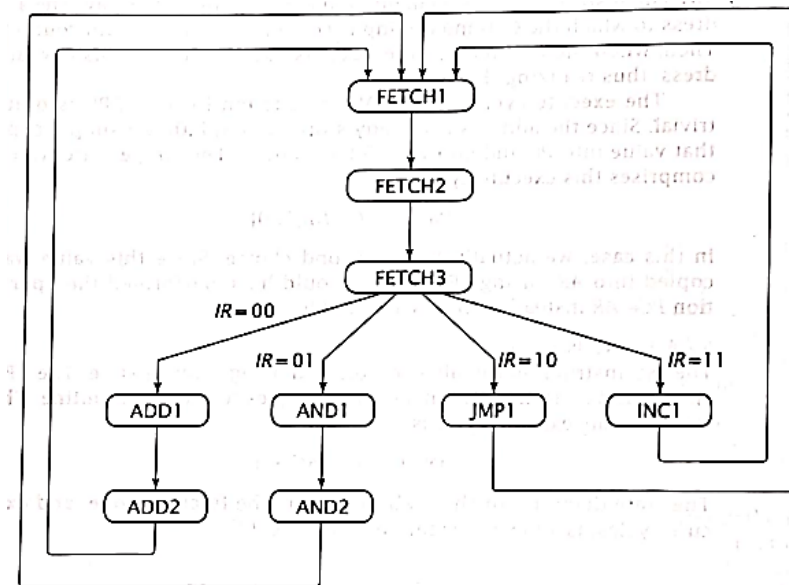
INC1: $AC \leftarrow AC + 1$

Establishing required Data paths

Instruction	Instruction Code	Operation
ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	$AC \leftarrow AC + 1$

- The state diagram and register transfers specify what must be done in order to realize this CPU.
- Now we must design the CPU so that it actually does these things.
- First, we look at what data transfers can take place and design the internal data paths of the CPU so this can be done.**
- The operations associated with each state for this CPU are:

FETCH1: $AR \leftarrow PC$
 FETCH2: $DR \leftarrow M, PC \leftarrow PC + 1$
 FETCH3: $IR \leftarrow DR[7..6], AR \leftarrow DR[5..0]$
 ADD1: $DR \leftarrow M$
 ADD2: $AC \leftarrow AC + DR$
 AND1: $DR \leftarrow M$
 AND2: $AC \leftarrow AC \wedge DR$
 JMP1: $PC \leftarrow DR[5..0]$
 INC1: $AC \leftarrow AC + 1$



- To design the data paths, we can take one of two approaches.
 - Create **direct paths** between each pair of components that transfer data.
 - Create a **bus** within the CPU and route data between components via the bus.
- Now we regroup the operations, without regard for the cycles in which they occur, by the register whose contents they modify. This result in the following:

```

FETCH1: AR←PC
FETCH2: DR←M, PC←PC + 1
FETCH3: IR←DR[7..6], AR←DR[5..0]
ADD1: DR←M
ADD2: AC←AC + DR
AND1: DR←M
AND2: AC←AC ^ DR
JMP1: PC←DR[5..0]
INC1: AC←AC + 1

```



```

AR:  AR←PC; AR←DR[5..0]
PC:  PC←PC + 1; PC←DR[5..0]
DR:  DR←M
IR:  IR←DR[7..6]
AC:  AC←AC + DR; AC←AC ^ DR; AC←AC + 1

```

AR: $AR \leftarrow PC$; $AR \leftarrow DR[5..0]$

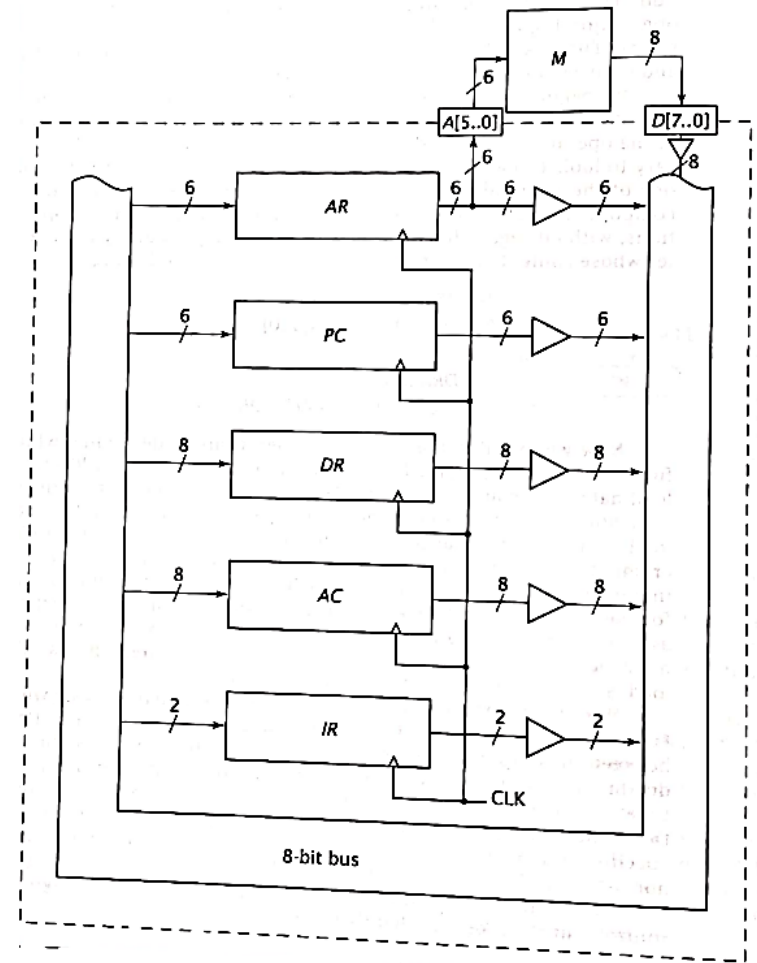
PC: $PC \leftarrow PC + 1$; $PC \leftarrow DR[5..0]$

DR: $DR \leftarrow M$

IR: $IR \leftarrow DR[7..6]$

AC: $AC \leftarrow AC + DR$; $AC \leftarrow AC \wedge DR$; $AC \leftarrow AC + 1$

- AR, DR and IR always load data from some other component.
- We connect every component to the system bus, as shown in figure.
- We **include tri-state buffers** to restrict data being placed all the times by everyone.
- Also, the output of AR are connected to pins A[5..0], as required in the CPU specification.



Preliminary register section for the Very Simple CPU

AR: $AR \leftarrow PC$; $AR \leftarrow DR[5..0]$

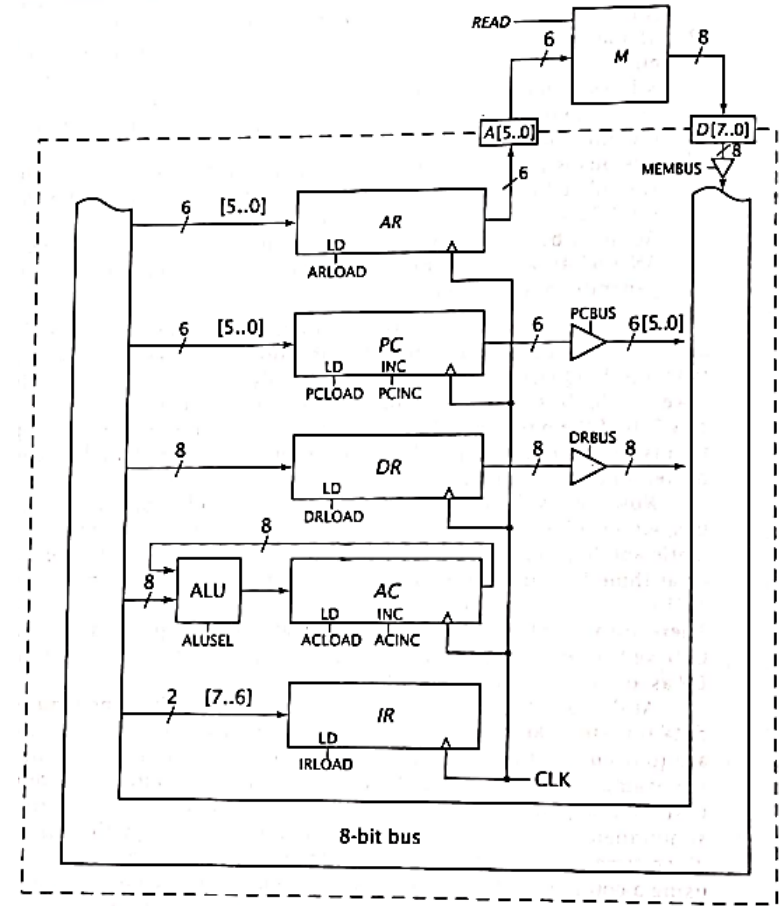
PC: $PC \leftarrow PC + 1$; $PC \leftarrow DR[5..0]$

DR: $DR \leftarrow M$

IR: $IR \leftarrow DR[7..6]$

AC: $AC \leftarrow AC + DR$; $AC \leftarrow AC \wedge DR$; $AC \leftarrow AC + 1$

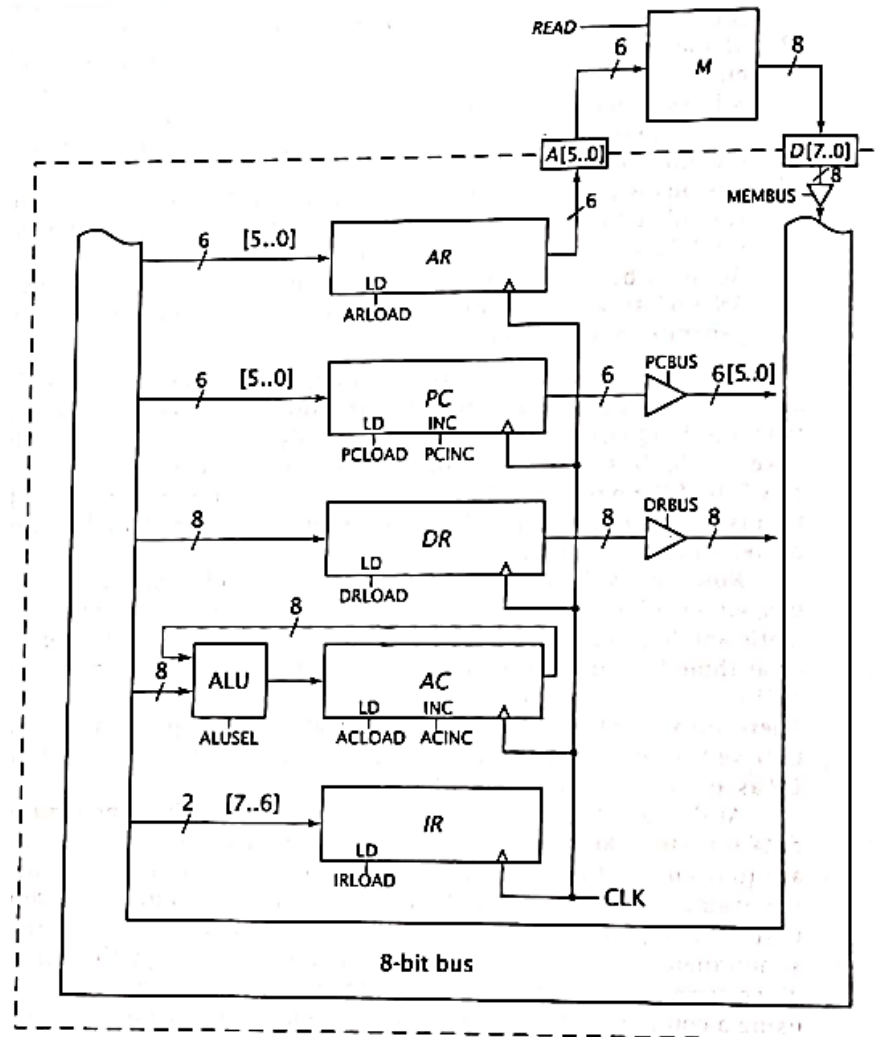
- AC can load in one of two values, either $AC+DR$ or $AC \wedge DR$.
- The CPU must incorporate some arithmetic and logic circuitry to generate these values.
 - When $ALUSEL = 0$, the output of ALU is arithmetic sum of its input, (i.e. $AC+DR$)
 - When $ALUSEL = 1$, the output of ALU is logical AND of its input. (i.e. $AC \wedge DR$)



Final register section for the Very Simple CPU

4.5 ALU Design

- The circuit diagram for the relatively simple CPU is shown in figure below:



ALU for relatively simple CPU

- The ALU for this CPU performs only two functions:
 - Adds its two inputs or
 - logically ANDs its two inputs.
- The simplest way to design this ALU is to create separate hardware to perform each function and then use a multiplexer to output one of the two results.
 - The addition is implemented using a standard 8-bit parallel adder.
 - The logical AND operation is implemented using eight bits 2-input AND gates.
 - The outputs of the parallel adder and the AND gates are input to an 8-bit 2 to 1 multiplexer.
 - The control unit of the MUX is called S (for select).

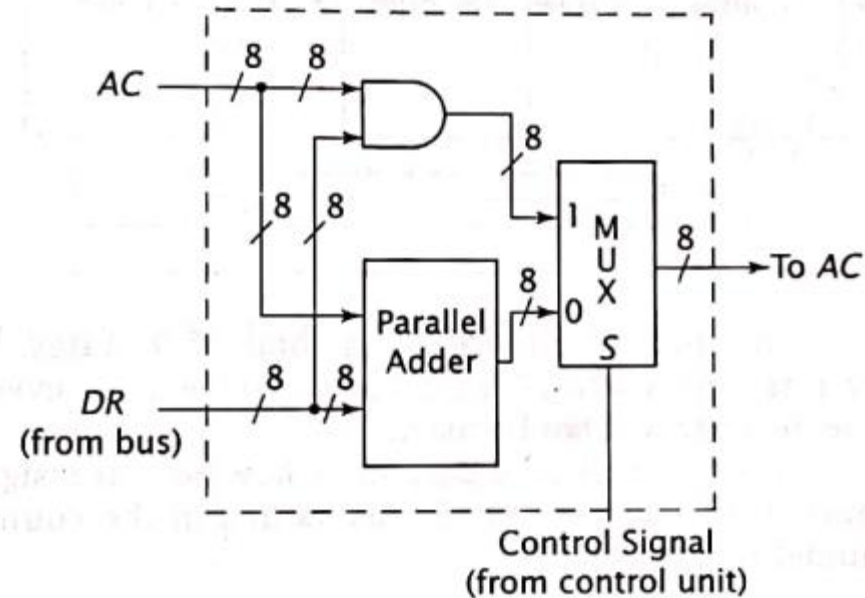
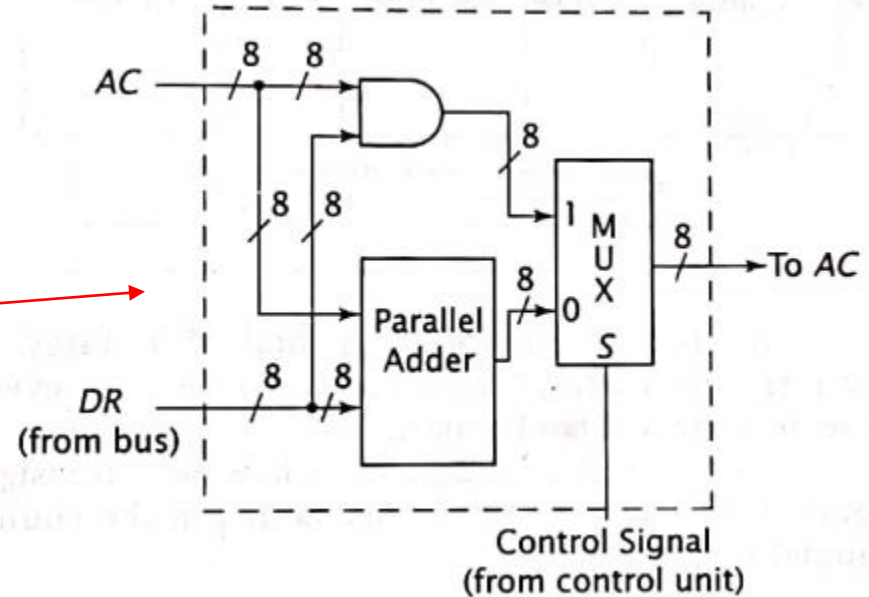
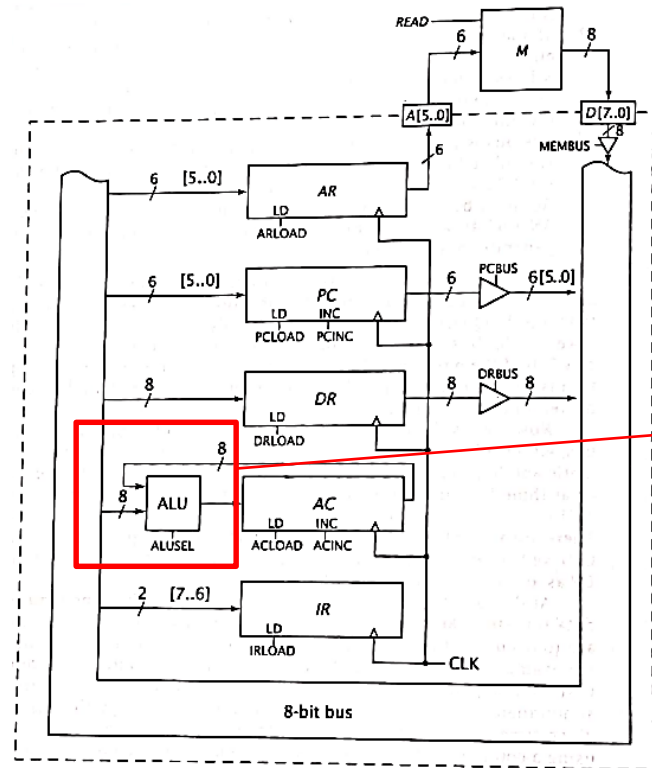


Figure: ALU Design



Summary:

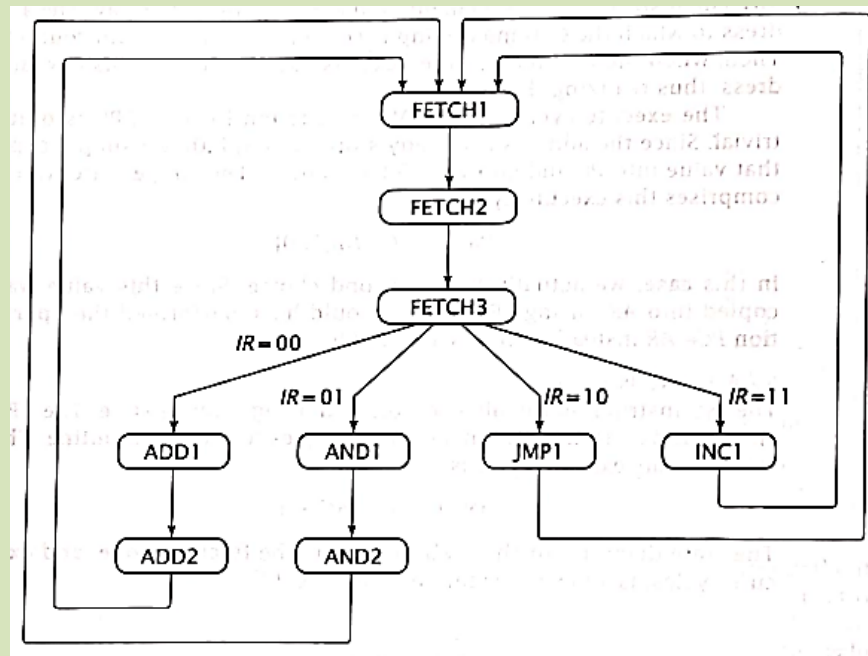
- The state diagram and RTL code for the micro-operation for our required CPU is as shown below:

RTL Code:

```
FETCH1: AR ← PC
FETCH2: DR ← M, PC ← PC + 1
FETCH3: IR ← DR[7..6], AR ← DR[5..0]
ADD1: DR ← M
ADD2: AC ← AC + DR
AND1: DR ← M
AND2: AC ← AC ∧ DR
JMP1: PC ← DR[5..0]
INC1: AC ← AC + 1
```

Instruction	Instruction Code	Operation
ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	$AC \leftarrow AC + 1$

State Diagram:



Summary (Contd.)

- After regrouping the operations, the RTL codes are:

```

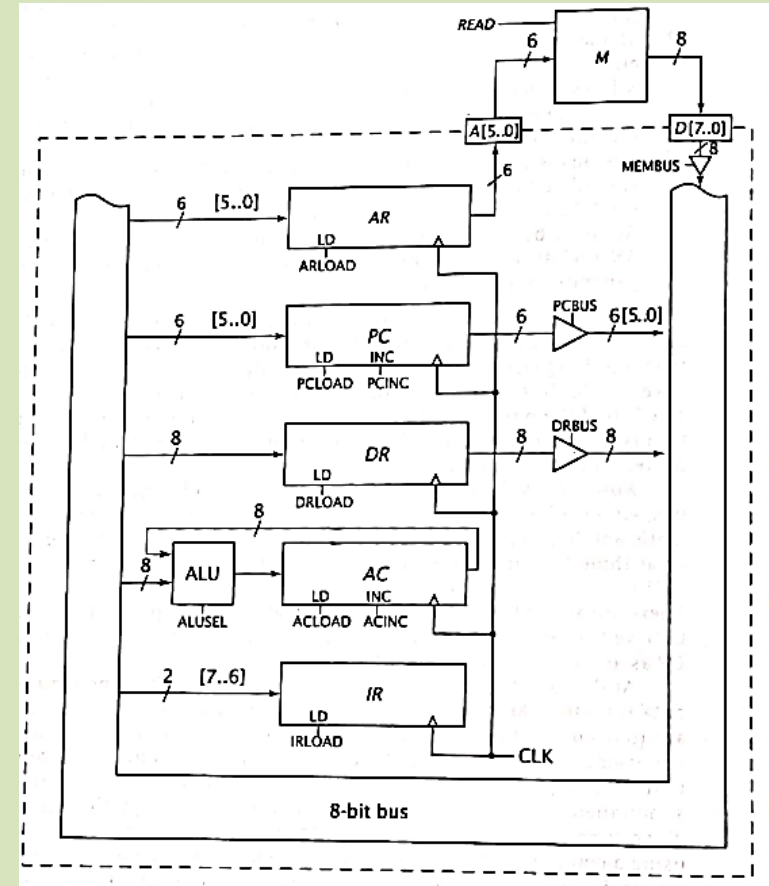
FETCH1: AR ← PC
FETCH2: DR ← M, PC ← PC + 1
FETCH3: IR ← DR[7..6], AR ← DR[5..0]
ADD1:  DR ← M
ADD2:  AC ← AC + DR
AND1:  DR ← M
AND2:  AC ← AC ^ DR
JMP1:  PC ← DR[5..0]
INC1:  AC ← AC + 1
    
```



```

AR:  AR ← PC; AR ← DR[5..0]
PC:  PC ← PC + 1; PC ← DR[5..0]
DR:  DR ← M
IR:  IR ← DR[7..6]
AC:  AC ← AC + DR; AC ← AC ^ DR; AC ← AC + 1
    
```

- Considering the regrouped RTL codes, our required register organization and datapath is as shown in the figure aside.



Summary (Contd.)

- The ALU for this CPU is as shown below:

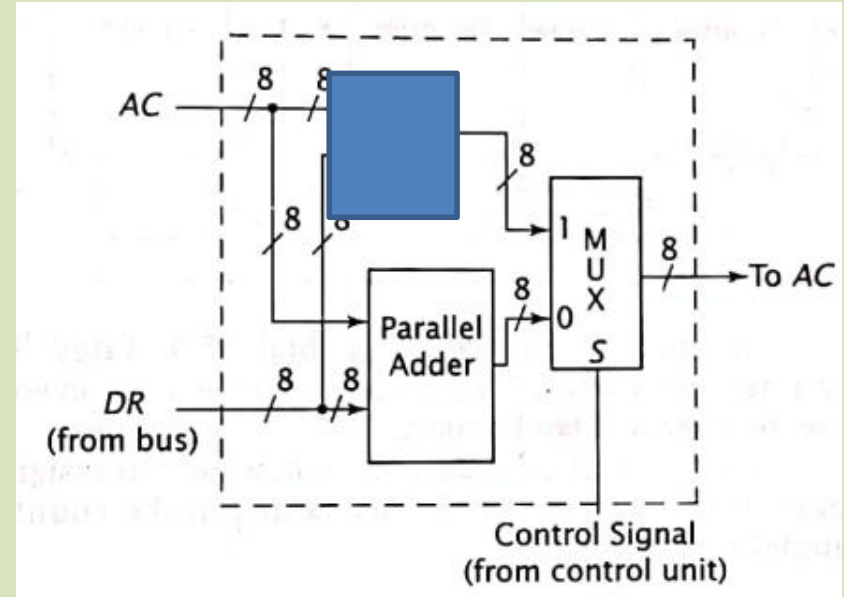


Figure: ALU Design

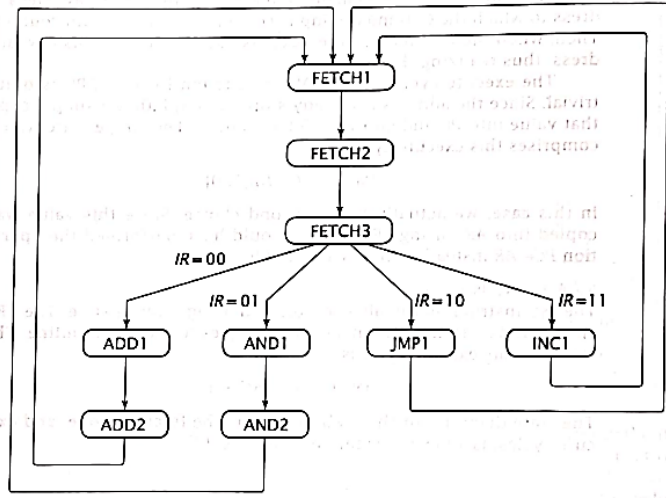
4.6 Designing Hardwired Control Unit

- After designing the CPU for every operation necessary to fetch, decode and execute the entire instruction set, the next step is **to design the circuit to generate control signals to cause operation to occur in proper sequence**. This is control unit.
- There are two methods:
 - Hardwired control
 - Microprogrammed control
- Hardwired control:
 - It uses sequential and combinational logic to generate control signals.
- Microprogrammed control:
 - It uses a lookup memory to output the control signals.

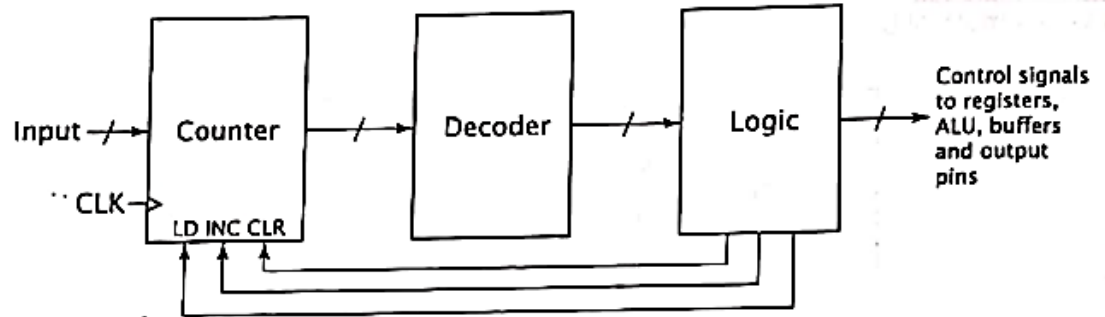
4.6 Designing Hardwired Control Unit (Contd.)

- Here, we focus on **hardwired control**.
- The simple control unit has three components:
 - a) **Counter:** it contains current state.
 - b) **Decoder:** It takes current state and generates individual signals for each state.
 - c) **Combinational Logic:** It takes the individual state signals and generate the control signals for each component as well as the signals to control the counter.
- These signals cause control unit to traverse the states in proper order.

Generic Hardwired Control Unit



- The CPU that we have designed has **9 total states**. So, a 4-bit counter and 4:16 decoder are needed. Here, seven of the outputs of the decode will not be used.
- **Input:** in input, states are provided.
- **CLK:** a clock signal
- **CLR:** CLR input is used to reach the state needed.
- **INC:** use INC input of counter to traverse the states.
- **LD:** use LD input to reach the proper execute routine.



Mapping Logic

- To make the function simple as possible:
- Consider the possible mapping 10[IR] i.e. if IR=00, the input to counter is 1000. For IR=01, the input is 1001 and so on.
- So, the counter values will be:

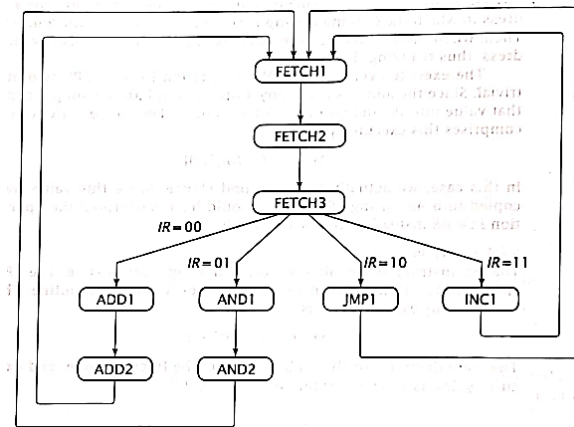
IR	Counter value	state
00	1000 (8)	ADD1
01	1001 (9)	AND1
10	1010 (10)	JMP1
11	1011 (11)	INC1

Table: Counter values for the proposed mapping function

- Here, the **problem arises**.
 - The **ADD1** and **ADD2** operations cannot be performed sequentially.
 - Since state **ADD1** has a counter value of 8 and state **AND1** has a counter value of 9, there is no counter value that has to be assigned to ADD2. (not best solution)
- So, we look for alternative solution.

For Reference:

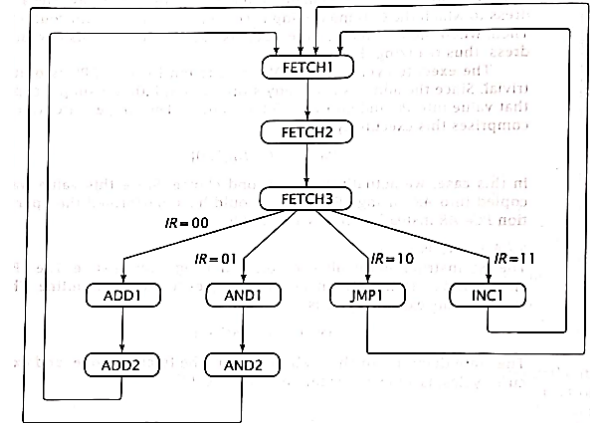
Instruction	Instruction Code	Operation
ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	$AC \leftarrow AC + 1$



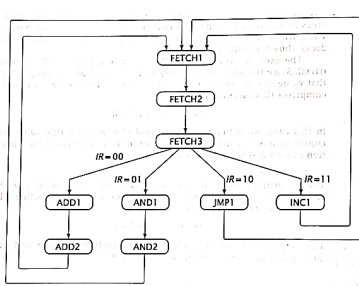
FETCH1: $AR \leftarrow PC$
 FETCH2: $DR \leftarrow M, PC \leftarrow PC + 1$
 FETCH3: $IR \leftarrow DR[7..6], AR \leftarrow DR[5..0]$
 ADD1: $DR \leftarrow M$
 ADD2: $AC \leftarrow AC + DR$
 AND1: $DR \leftarrow M$
 AND2: $AC \leftarrow AC \wedge DR$
 JMP1: $PC \leftarrow DR[5..0]$
 INC1: $AC \leftarrow AC + 1$

- Now if we mapping function as: 1[IR]0, which results in counter values of:

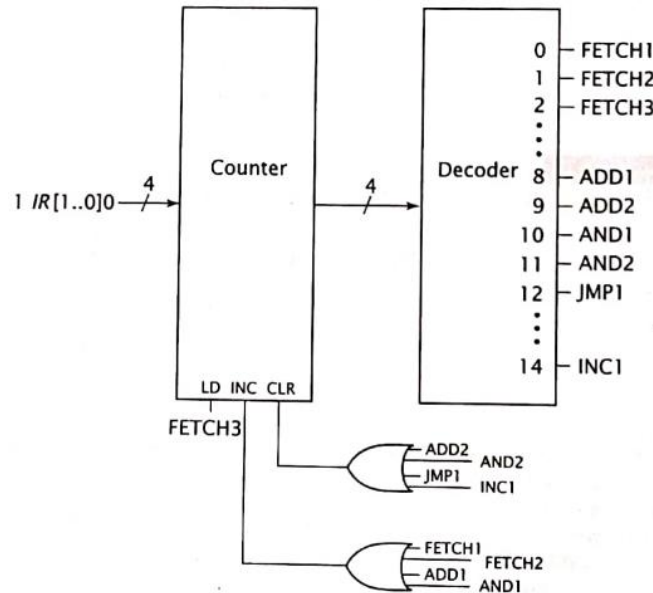
IR	Counter	Value	state
00	1 <u>00</u> 0 (8)	8	ADD1
01	1 <u>01</u> 0 (10)	10	AND1
10	1 <u>10</u> 0 (12)	12	JMP1
11	1 <u>11</u> 0 (14)	14	INC1



- Now, we can assign counter value 9 to ADD2 and 11 to AND2
- This enables to perform sequentially.



- So, a complete hardwired control unit is:



- For Counter, INC, CLR and LD signals are needed.
- INC is asserted when control unit is traversing sequential states, during Fetch1, Fetch2, JMP1 and INC1
- LD is asserted at end of fetch cycle during state Fetch3.
- CLR is asserted at the end of each execute cycle to return to fetch cycle. This happens during ADD2, AND2, JMP1 and INC1

Summary:

Our CPU's state diagram is as shown below:

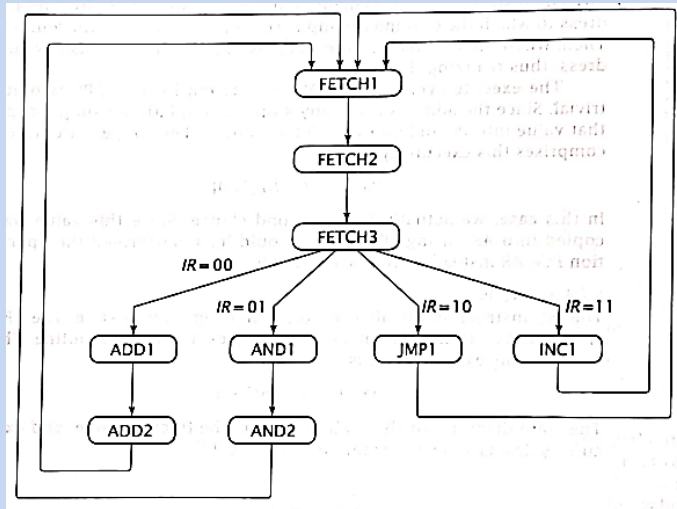


Figure: State diagram

- The CPU that we have designed has **9 total states**. So, a 4-bit counter and 4:16 decoder are needed. Here, seven of the outputs of the decode will not be used.
- The general block diagram for our required Hardwired Control Unit is as given below:

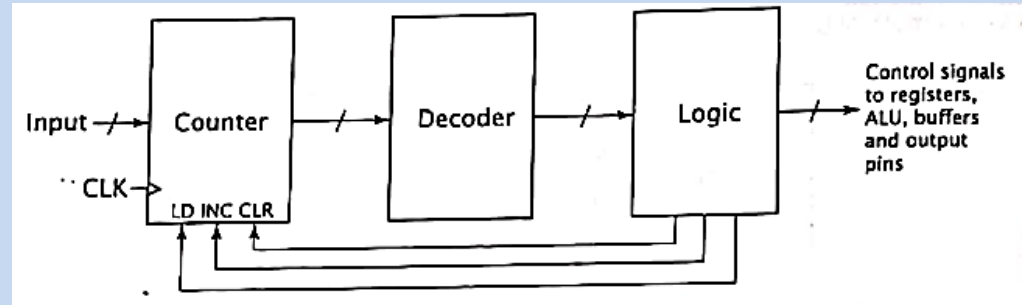


Figure: General Block diagram of Control Unit

Summary (contd.)

- Now if we mapping function as: $1[IR]0$, which results in counter values of:

IR	Counter	Value	state
00	1 <u>00</u> 0 (8)	8	ADD1
01	1 <u>01</u> 0 (10)	10	AND1
10	1 <u>10</u> 0 (12)	12	JMP1
11	1 <u>11</u> 0 (14)	14	INC1

- Now, we can assign counter value 9 to ADD2 and 11 to AND2
- This enables to perform sequentially.
- So, a complete hardwired control unit is:

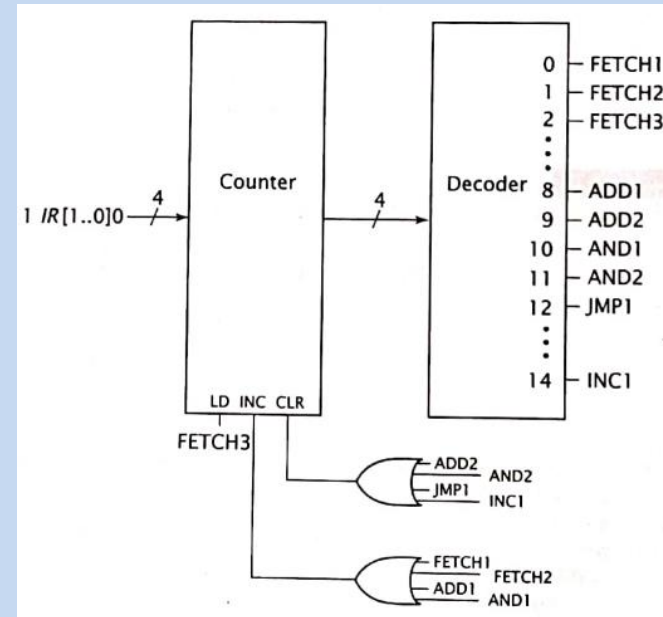


Figure: Hardwired Unit for our CPU

Some Few Worked out Examples:

Example 1:

Design CPU for below:

Instruction	Instruction Code	Operation
JMP1	00AAAAAA	$PC \leftarrow AAAAAA + 1$
INC2	01XXXXXX	$AC \leftarrow AC + 2$
ADD1	10AAAAAA	$AC \leftarrow AC + M[AAAAAA] + 1$
SKIP	11XXXXXX	$PC \leftarrow PC + 1$

Solution:

The state diagram and RTL code for the micro-operation for our required CPU is as shown below:

Here, micro-operations are:

- Fetch 1: $AR \leftarrow PC$
- Fetch 2: $DR \leftarrow M, \quad PC \leftarrow PC + 1$
- Fetch 3: $IR \leftarrow DR[7,6], \quad AR \leftarrow DR[5...0]$
- JMP11: $PC \leftarrow AR$
- JMP12: $PC \leftarrow PC + 1$
- INC21: $AC \leftarrow AC + 1$
- INC22: $AC \leftarrow AC + 1$
- ADD11: $DR \leftarrow M, \quad AC \leftarrow AC + 1$
- ADD12: $AC \leftarrow AC + DR$
- SKIP1: $PC \leftarrow PC + 1$

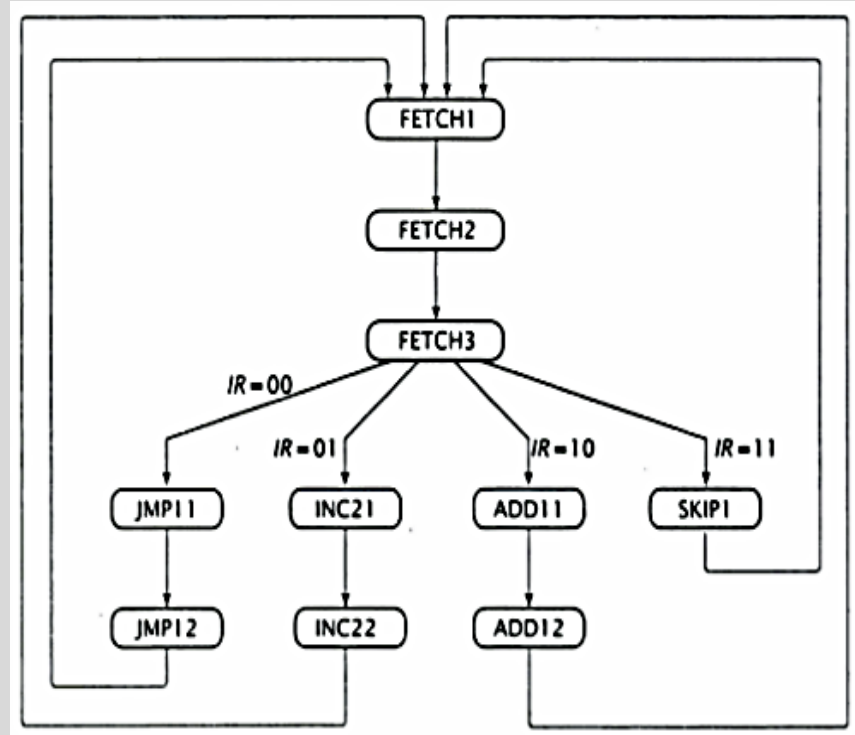


Figure: State diagram

- After regrouping the operations, the RTL codes are:

ALU section:

- The ALU for this CPU is as shown below:

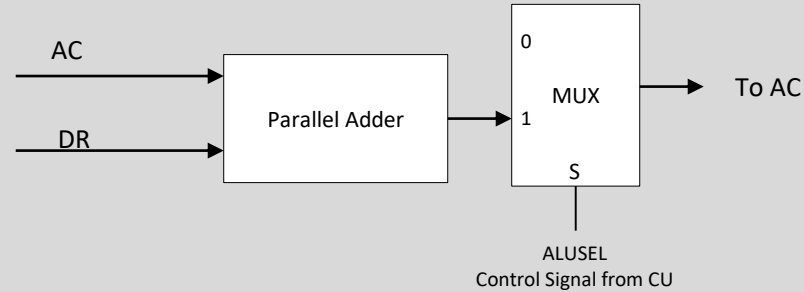
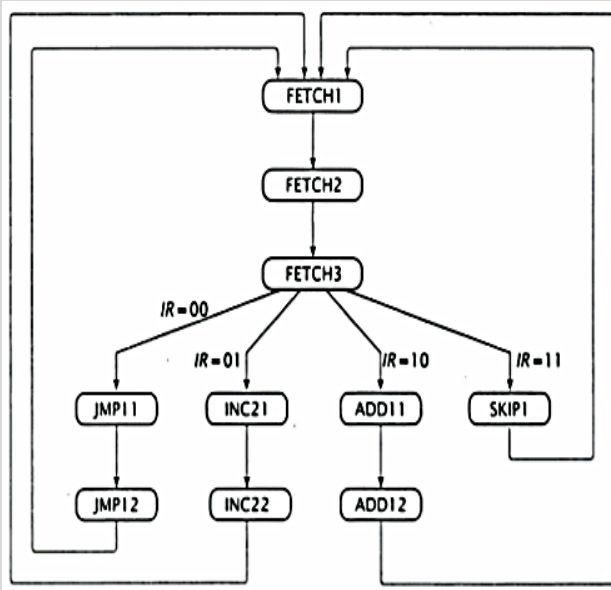


Figure: ALU Design

Hardware Unit

Our CPU's state diagram is as shown below:



- The CPU that we have designed has **10 total states**. So, a 4-bit counter and 4:16 decoder are needed. Here, six of the outputs of the decode will not be used.
- The general block diagram for our required Hardwired Control Unit is as given below:

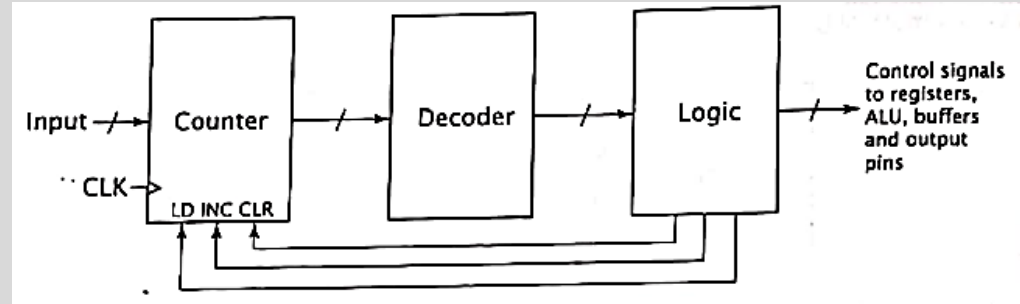


Figure: General Block diagram of Control Unit

- Now if we mapping function as: $1[IR]0$, which results in counter values of:

IR	Counter	Value	state
00	1 <u>00</u> 0 (8)	8	JMP11
01	1 <u>01</u> 0 (10)	10	INC21
10	1 <u>10</u> 0 (12)	12	ADD11
11	1 <u>11</u> 0 (14)	14	SKIP1

- Now, we can assign counter value 9 to JMP12 and 11 to INC22
- This enables to perform sequentially.
- So, a complete hardwired control unit is:

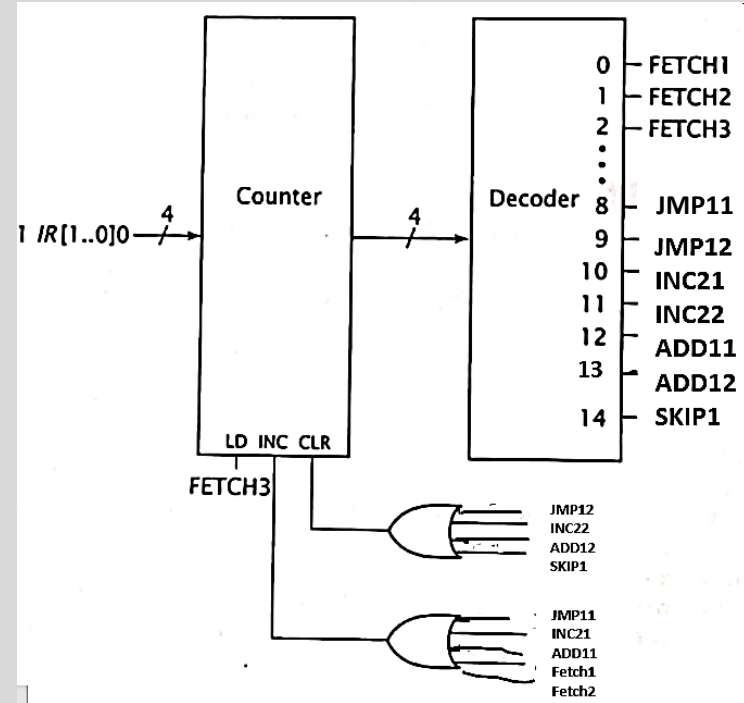


Figure: Hardwired Unit for our CPU

Example 2: Design a CPU that meets the following specifications.

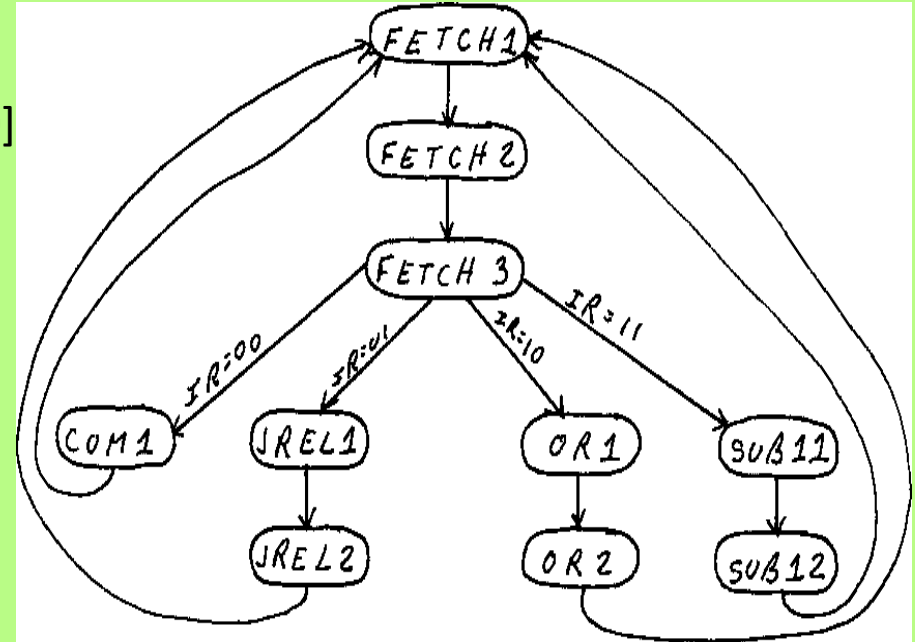
- It can access 64 words of memory, each word being 8 bits wide. The CPU does this by outputting a 6-bit address on its output pins A[5..0] and reading in the 8-bit value from memory on its inputs D[7..0].
- The CPU contains a 6-bit address register (AR) and program counter (PC); an 8-bit accumulator (AC) and data register (DR); and a 2-bit instruction register (IR).
- The CPU must realize the following instruction set.

Instruction	Instruction Code	Operation
COM	00XXXXXX	$AC \leftarrow AC'$
JREL	01AAAAAA	$PC \leftarrow PC + 00AAAAAA$
OR	10AAAAAA	$AC \leftarrow AC \vee M[00AAAAAA]$
SUB1	11AAAAAA	$AC \leftarrow AC - M[00AAAAAA] - 1$

Solution:

State diagram and RTL code:

- FETCH1: $AR \leftarrow PC$
- FETCH2: $DR \leftarrow M, PC \leftarrow PC + 1$
- FETCH3: $IR \leftarrow DR[7..6], AR \leftarrow DR[5..0]$
- COM1: $AC \leftarrow AC'$
- JREL1: $DR \leftarrow M$
- JREL2: $PC \leftarrow PC + DR[5..0]$
- OR1: $DR \leftarrow M$
- OR2: $AC \leftarrow AC \cup DR$
- SUB11: $DR \leftarrow M$
- SUB12: $AC \leftarrow AC + DR'$



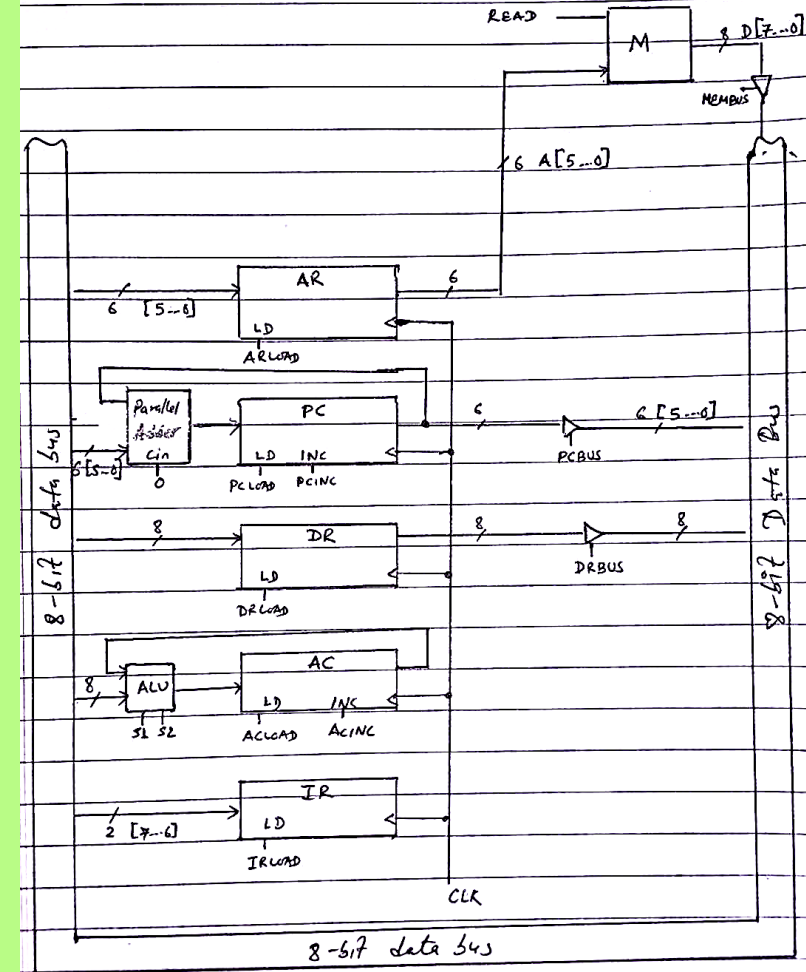
- After regrouping the operations, the RTL codes are:

- $\text{FETCH1} : AR \leftarrow PC$
- $\text{FETCH2} : DR \leftarrow M, PC \leftarrow PC + 1$
- $\text{FETCH3} : IR \leftarrow DR[7..6], AR \leftarrow DR[5..0]$
- $\text{COM1} : AC \leftarrow AC'$
- $\text{JREL1} : DR \leftarrow M$
- $\text{JREL2} : PC \leftarrow PC + DR[5..0]$
- $\text{OR1} : DR \leftarrow M$
- $\text{OR2} : AC \leftarrow AC \cup DR$
- $\text{SUB11} : DR \leftarrow M$
- $\text{SUB12} : AC \leftarrow AC + DR'$

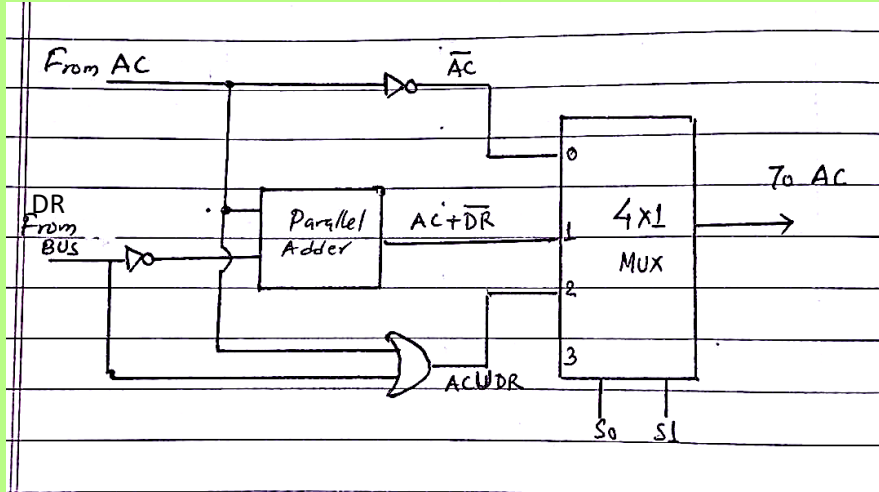


- AR: $AR \leftarrow PC$, $AR \leftarrow DR[5..0]$
- PC: $PC \leftarrow PC + 1$, $PC \leftarrow PC + DR[5..0]$
- DR: $DR \leftarrow M$
- AC: $AC \leftarrow AC'$, $AC \leftarrow AC \cup DR$, $AC \leftarrow AC + DR'$
- IR: $IR \leftarrow DR[7,6]$

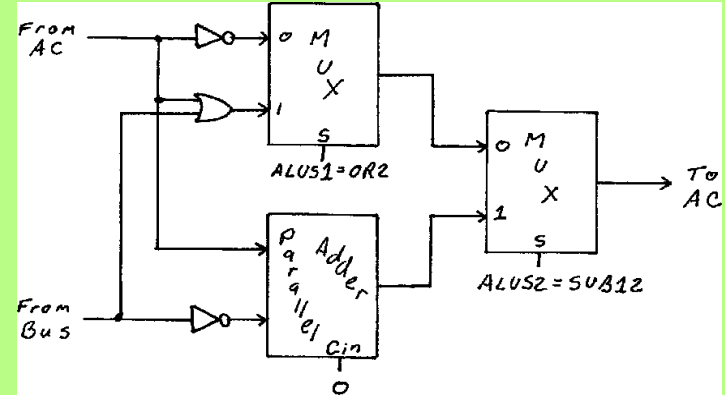
- Considering the regrouped RTL codes, our required register organization and datapath is as shown in the figure aside.



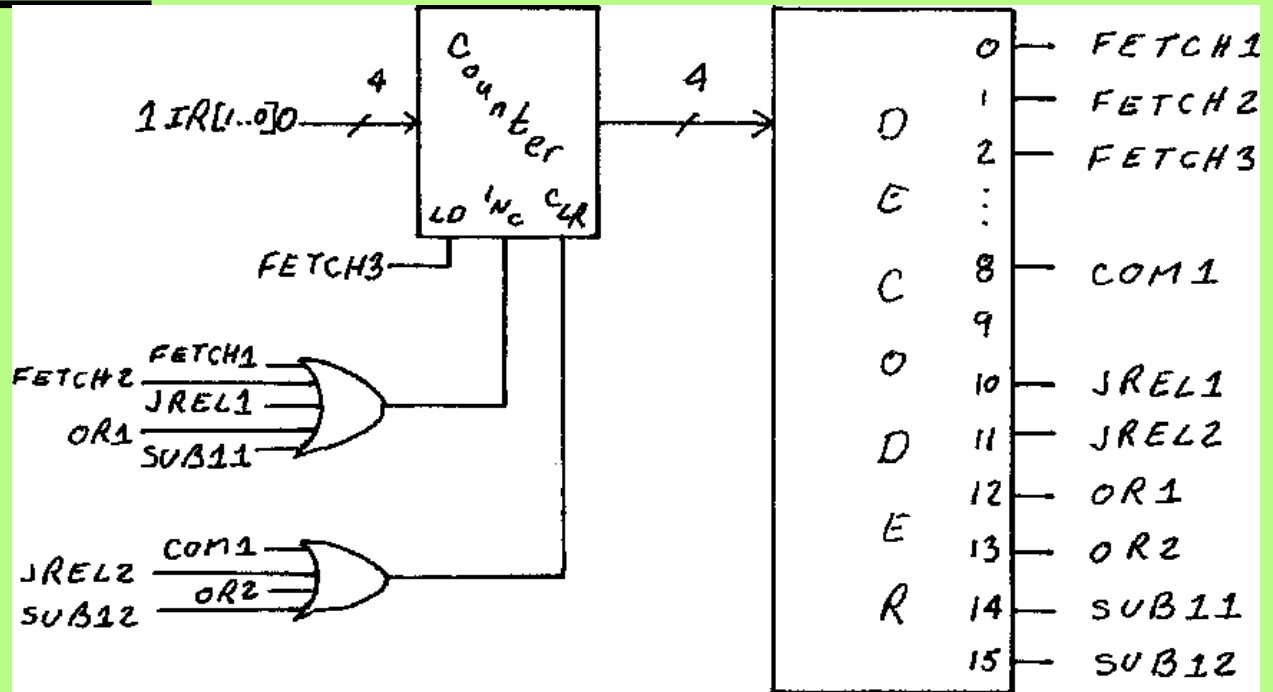
ALU section:



OR



Control unit:



Note: Write short explanation as before.

Example 3:

- Design a CPU that has 6-bit address register (AR), 6-bit program counter (PC), 8-bit data register (DR) and 2-bit instruction register (IR). The CPU must execute the following instructions:

Instruction	Instruction code	Operation
COM	00xxxxxx	$AC \leftarrow \overline{AC}$
AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JREL	10AAAAAA	$PC \leftarrow PC + AAAAAA$
SKIP	11xxxxxx	$PC \leftarrow PC + 1$

- Show the RTL code for fetch and execute cycles for each instruction and draw state diagram.
- Show the final register section
- Show the ALU Design
- Show hardwired control unit for same CPU.

Solution:

- RTL code and State diagram

Fetch1 : $AR \leftarrow PC$

Fetch2 : $DR \leftarrow M, PC \leftarrow PC+1$

Fetch3 : $IR \leftarrow DR[7-6], AR \leftarrow DR[5..0]$

COM1 : $AC \leftarrow AC'$

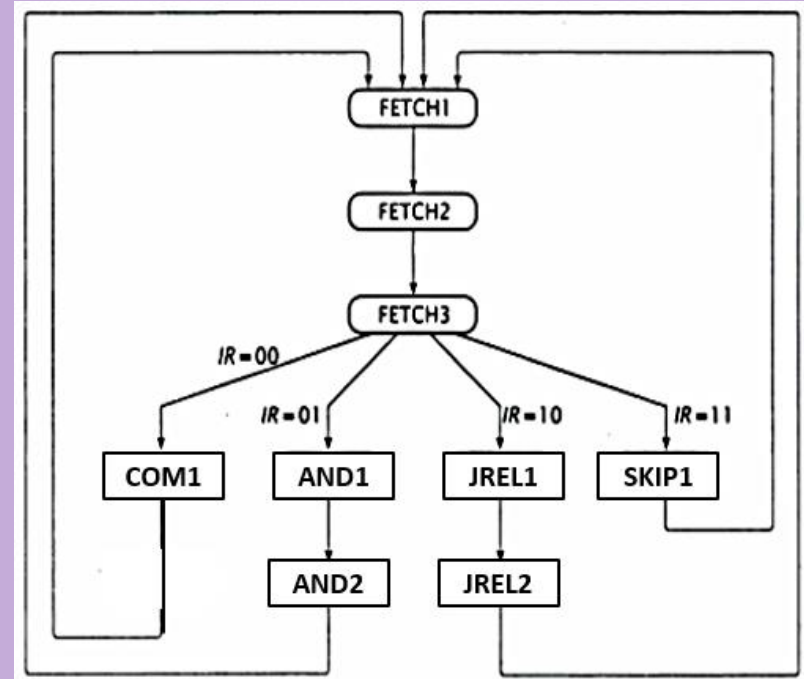
AND1 : $DR \leftarrow M$

AND2 : $AC \leftarrow AC \wedge DR$

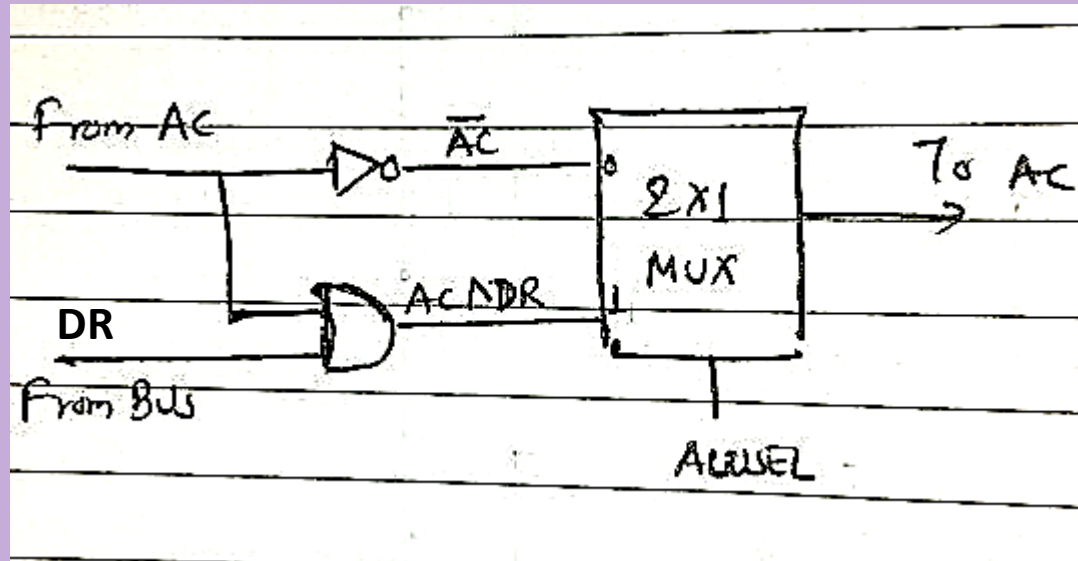
JREL1 : $AR \leftarrow DR[5..0]$

JREL2 : $PC \leftarrow PC + AR$

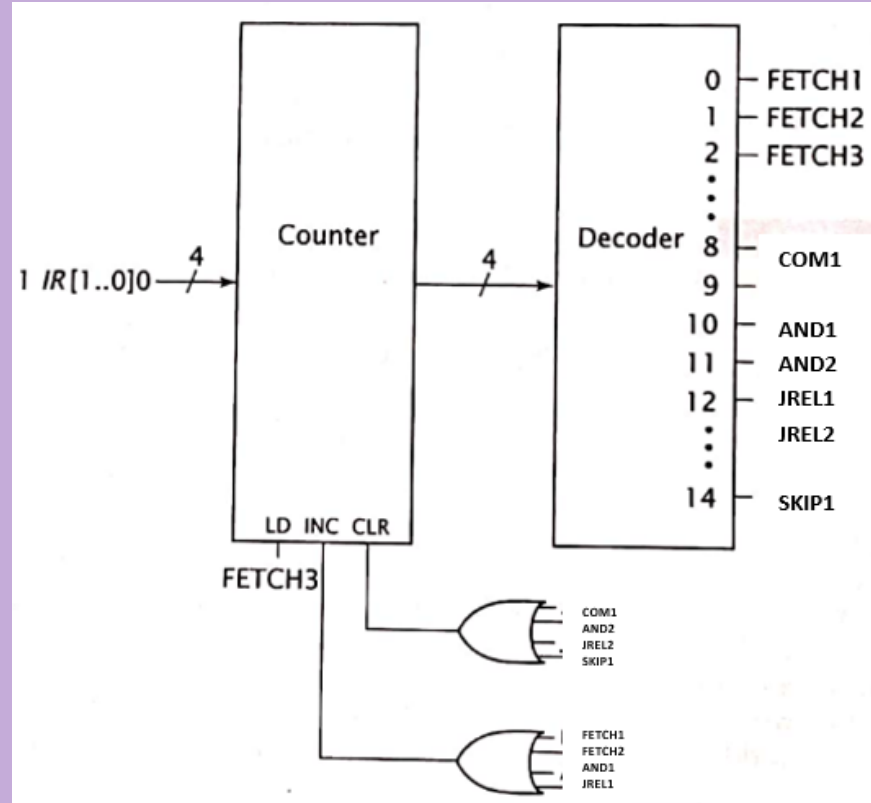
SKIP1 : $PC \leftarrow PC+1$



ALU section:



Control unit:



Note: Write short explanation as before.

Exam Questions:

Exam Questions:

1. a) **There is a Very Simple CPU for the given set of Instructions: [PU 2017 Fall]**
- LDA (Load Accumulator with a value from memory) with opcode 00;
 - AND (performing and operation to the content of accumulator with the value at some memory address) with opcode 01;
 - SKIP (skipping an instruction) with opcode 10;
 - DEC (decrementing the value at accumulator) with opcode 11;

Let the instruction width be 8 bits and address is 6 bits. Design the CPU's Register section, state diagram and ALU.

- b) **Design the hardwired control unit for the CPU described in above question. [PU 2017 Fall]**

2. a) There is a very simple CPU for the given set of Instruction: [PU 2017 Spring]

Instruction	Instruction Code	Micro-operation
ADD	00 AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
SUB	01 AAAAAA	$AC \leftarrow AC - M[AAAAAA]$
SKIP	10 XXXXXX	$PC \leftarrow PC + 1$
DEC2	11 XXXXXX	$AC \leftarrow AC - 2$

Let the instruction width be 8 bits and address is 6 bits. Design the CPU's register section, state diagram and ALU.

- b) Design the hardwired control unit for the CPU described in above question. [PU 2017 Spring]

3. a) There is a very simple CPU for the given set of Instruction:
[PU 2018 Fall]

Instruction	Instruction Code	Operations
STA	00 AAAAAA	$M[AAAAAA] \leftarrow AC$
XNOR	01 AAAAAA	$AC \leftarrow AC \oplus M[AAAAAA]$
JMP	10 AAAAAA	GOTO AAAAAA
SKIP	11 XXXXXX	$PC \leftarrow PC + 1$

Let the instruction width be 8 bits and address is 6 bits. Design the CPU's register section, state diagram and ALU.

- b) Design the hardwired control unit for the CPU described in above question. [PU 2018 Fall]

4. a) There is a very simple CPU for the given set of Instruction:
[PU 2018 Spring]

Instruction	Instruction Code	Operations
STR	00 AAAAAA	$AC \rightarrow M[AAAAAA]$
NAND	01 AAAAAA	$AC \leftarrow (AC \wedge M[AAAAAA])'$
JMP	10 AAAAAA	$PC \leftarrow AAAAAA$
INC2	11 XXXXXX	$AC \leftarrow AC + 2$

Let the instruction width be 8 bits and address is 6 bits. Design the CPU's register section, state diagram and ALU.

b) Design the hardwired control unit for the CPU described in above question. [PU 2018 Spring]

5. a) There is a very simple CPU for the given set of Instruction: [PU 2019 Spring]

Very simple CPU for the given set of instructions:

Instruction	Instruction Code	Operations
STR	00 AAAAAA	$AC \rightarrow M[AAAAAA]$
AND	01 AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
JMPR	10 AAAAAA	$PC \leftarrow PC + AAAAAA$
DEC2	11 XXXXXX	$AC \leftarrow AC - 2$

Let the instruction width be 8 bits and address is 6 bits. Design the CPU's register section, state diagram and ALU.

- b) Design the hardwired control unit for the CPU described in above question. [PU 2019 Spring]

6. For a very simple CPU, has the following instruction set. Show the state diagram, register section, ALU design.
[PU 2020 Spring]

Operation	Instruction Code	Instruction
$M[AAAAAA] \leftarrow AC$	00AAAAAA	STA
$AC \leftarrow M[AAAAAA]$	01AAAAAA	LDA
$AC \leftarrow AC + 2$	10XXXXXX	INC
$AC \leftarrow AC'$	11 XXXXXX	COM

End of chapter 4